# Chapter 1

# Interfaces, Free Surfaces and Surface Transport: Theory and implementation

This document provides the theoretical background underlying `oomph-lib's` free surface, interface and surface transport capabilities. We begin with a review of the relevant theory to establish the overall framework and notation, and then discuss the implementation of the methodology in `oomph-lib`.

Here is an overview of the structure of this document:


- Theory
  - Geometry of Surfaces
  - Differential Operators On A Surface
  - Free Surface and Interface Boundary Conditions
  - Surface Transport Equations
- Implementation
  - The FluidInterfaceElement class
  - The LineDerivatives class
  - The AxisymmetricDerivatives class
  - The SpineLine/Axi/SurfaceFluidInterfaceElement classes
  - The ElasticLine/Axi/SurfaceFluidInterfaceElement classes


If you don't need all the gory details, you may prefer to start by exploring the free-surface flow tutorials in `oomph-lib's list of example driver codes.`

## 1.1 Theory

A complete theoretical treatment of moving surfaces in space requires the use of differential geometry, described in detail by Aris (1962) *Vectors, Tensors and the Basic Equations of Fluid Mechanics* , Wetherburn (1955) *Differential Geometry of Three Dimensions, Volume I* and Green & Zerna (1968) *Theoretical Elasticity* among others. Here, we shall present only the essential information and the reader is referred to these texts to fill in additional background.

### 1.1.1 Geometry of Surfaces

For our purposes, a surface is a object of dimension $n-1$ embedded in an $n$-dimensional Euclidean space, where $n$ is two or three. We can always represent such a surface by a position vector from our chosen origin $\mathbf{R}^*(\zeta^\alpha, t)$, parametrised by time $t$ and intrinsic (surface) coordinates $\zeta^\alpha$, where the Greek index $\alpha = 1, .., n-1$.
Throughout this document we will use the summation convention that repeated Roman indices are to be summed over the range from 1 to $n$ and repeated Greek indices are to be summed over the range from 1 to $n-1$. We will retain the summation signs for all other sums, such as sums over the nodes etc.
The covariant base vectors of the surface are defined to be the partial derivatives of the position vector with respect to the surface coordinates. For a two-dimensional surface,

$$\boldsymbol{g}_1 = \frac{\partial \boldsymbol{R}}{\partial \zeta^1}, \qquad \boldsymbol{g}_2 = \frac{\partial \boldsymbol{R}}{\partial \zeta^2}.$$

The covariant metric tensor of the surface is formed by taking the dot product of the covariant base vectors

$$g_{\alpha\beta} = \boldsymbol{g}_\alpha \cdot \boldsymbol{g}_\beta,$$

and the determinant of the covariant metric tensor is denoted by

$$g = g_{11} g_{22} - g_{12} g_{21}.$$

The contravariant metric tensor is the inverse of the covariant metric tensor and is denoted by $g^{\alpha\beta}$. Thus,

$$g^{11} = g_{22}/g, \quad g^{12} = -g_{12}/g, \quad g^{21} = -g_{21}/g, \quad g^{22} = g_{11}/g.$$

For a one-dimensional surface, there is a single covariant base vector that coincides with the tangent vector

$$\boldsymbol{g}_1 = \frac{\partial \boldsymbol{R}}{\partial \zeta^1}.$$

Here, the covariant metric tensor is simply the inner product of the tangent vector with itself $g_{11} = \boldsymbol{g}_1 \cdot \boldsymbol{g}_1$ with determinant $g = g_{11}$ and the contravariant metric tensor is $g^{11} = 1/g_{11}$.
We will need to integrate quantities over the surface, which uses the result that an infinitesimal unit of area is

$$\mathrm{d}S = \sqrt{g}\, \mathrm{d}\zeta^\alpha,$$

in terms of the intrinsic coordinates.

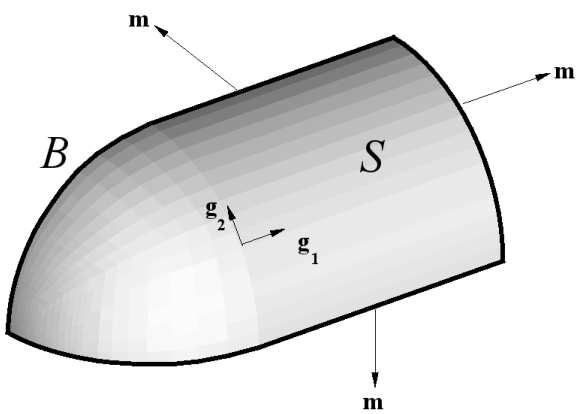


**Figure 1.1 Sketch of section of a two-dimensional surface, S , in a three-dimensional space. The covariant base vectors are tangent to the surface and in the direction of the intrinsic surface coordinates. The intrinsic coordinates parametrise the position vector to material points in the surface. The surface is bounded by the curve B and the vector m is perpendicular to the outer unit normal of the surface and tangent to the bounding curve.**

### 1.1.2 Differential Operators On A Surface

The formulation of transport equations within the surface requires the rates of change of surface quantities. The appropriate derivative is the surface gradient, often written as $\boldsymbol{\nabla}_s$ and defined unhelpfully in many papers and textbooks as the gradient operator restricted to the surface or

$$\boldsymbol{\nabla}_S\phi = \boldsymbol{\nabla}\phi - (\boldsymbol{\nabla}\phi\cdot\boldsymbol{n})\boldsymbol{n}.$$

The problem with the above definition is that if a quantity is defined only on the surface it is impossible to take its gradient $\boldsymbol{\nabla}$.
A more helpful definition in terms of the intrinsic coordinates is that

$$\boldsymbol{\nabla}_s\phi = g^{\alpha\beta}\boldsymbol{g}_\alpha\frac{\partial\phi}{\partial\zeta^\beta}.$$

For a one dimensional surface parametrised by the arc-length, $s$, this reduces to $\boldsymbol{\nabla}_s\phi = \boldsymbol{t}\partial\phi/\partial s$, where $t$ is a unit tangent vector to the surface. By definition the surface gradient is tangent to the surface and has no normal component.
The surface divergence of an $n$-dimensional vector quantity defined on the surface is given by

$$\boldsymbol{\nabla}_s\cdot\boldsymbol{v} = g^{\alpha\beta}\boldsymbol{g}_\alpha\cdot\frac{\partial\boldsymbol{v}}{\partial\zeta^\beta}.$$

The divergence theorem applied over the surface (Aris, 1955) is

$$\iint_S \boldsymbol{\nabla}_s\cdot\boldsymbol{v}_t\ \mathrm{d}S = \int_B \boldsymbol{v}_t\cdot\boldsymbol{m}\,\mathrm{d}l,$$

where $\boldsymbol{v}_t$ is a vector tangential to the surface; and $\boldsymbol{m}$ is the unit vector tangent to the surface, but perpendicular to the tangent of the bounding curve $B$, see the Figure above.
If a general vector is decomposed into normal and tangential components, $\boldsymbol{v} = \boldsymbol{v}_t + v_n\boldsymbol{n}$, we can write

$$\iint_S \boldsymbol{\nabla}_s\cdot\boldsymbol{v}\ \mathrm{d}S = \iint_S \boldsymbol{\nabla}_s\cdot(\boldsymbol{v}_t + v_n\boldsymbol{n})\ \mathrm{d}S = \iint_S \boldsymbol{\nabla}_s\cdot\boldsymbol{v}_t\ \mathrm{d}S + \iint_S \boldsymbol{\nabla}_s\cdot(v_n\boldsymbol{n})\ \mathrm{d}S$$

$$= \iint_S \boldsymbol{\nabla}_s\cdot\boldsymbol{v}_t\ \mathrm{d}S\ + \iint_S v_n\boldsymbol{\nabla}_s\cdot\boldsymbol{n} + \boldsymbol{n}\cdot\boldsymbol{\nabla}_s v_n\ \mathrm{d}S = \iint_S \boldsymbol{\nabla}_s\cdot\boldsymbol{v}_t\ \mathrm{d}S\ - \iint_S v_n\kappa\ \mathrm{d}S,$$

where $\kappa$ is twice the mean curvature of the surface and equal to minus the surface divergence of the normal. The term $\boldsymbol{n}\cdot\boldsymbol{\nabla}_s v_n = 0$ because the normal and surface divergence of any scalar quantity are orthogonal.
Hence using the divergence theorem on the first term on the right-hand side, we obtain

$$\iint_S \boldsymbol{\nabla}_s\cdot\boldsymbol{v}\ \mathrm{d}S = -\iint_S v_n\kappa\ \mathrm{d}S + \int_B \boldsymbol{v}_t\cdot\boldsymbol{m}\,\mathrm{d}l = -\iint_S v_n\kappa\ \mathrm{d}S + \int_B \boldsymbol{v}\cdot\boldsymbol{m}\,\mathrm{d}l, \quad (1)$$

because the normal component of $\boldsymbol{v}$ is perpendicular to $\boldsymbol{m}$ and therefore contributes nothing to the line integral.
If we now choose $\boldsymbol{v} = \phi\,\boldsymbol{e}_i$, where $\boldsymbol{e}_i$ is the unit base vector in the $i$-th Cartesian coordinate direction, then we obtain

$$\iint_S \boldsymbol{\nabla}_s\phi\cdot\boldsymbol{e}_i\ \mathrm{d}S\ = -\iint_S \phi(\boldsymbol{e}_i\cdot\boldsymbol{n})\kappa\ \mathrm{d}S + \int_B \phi\,\boldsymbol{e}_i\cdot\boldsymbol{m}\,\mathrm{d}l,$$

which is equivalent to the surface divergence theorem for a scalar field described by Wetherburn (1955; p 240, Eqn 26)

$$\iint_S \kappa\phi\boldsymbol{n}\,\mathrm{d}S = \int_B \phi\boldsymbol{m}\,\mathrm{d}l - \iint_S \boldsymbol{\nabla}_s\,\phi\,\mathrm{d}S. \qquad (2)$$

### 1.1.3 Free Surface and Interface Boundary Conditions

#### 1.1.3.1 Dynamic condition

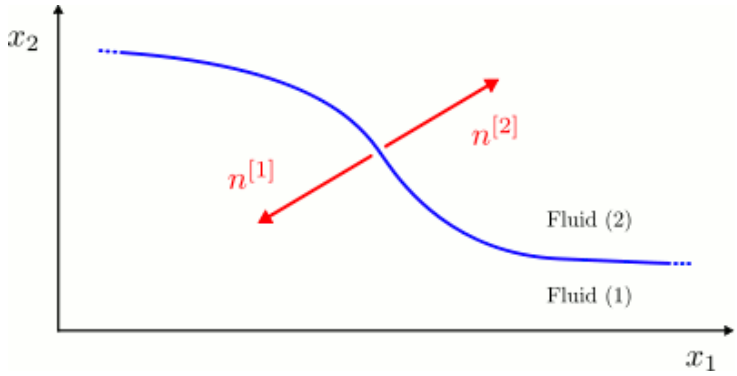


**Figure 1.2 Sketch of the interface between two fluids.**

The presence of an interface with non-constant surface tension $\sigma^*$ contributes to the overall force balance, as also described in another tutorial . The surface tension acts a line force bounding the interface and acting in the direction $\boldsymbol{m}$. Thus using the surface divergence theorem in the form (2),

$$\int_B \sigma^* \boldsymbol{m} \, \mathrm{d}l = \iint_S \sigma^* \kappa + \boldsymbol{\nabla}_s \, \sigma^* \, \mathrm{d}S,$$

which gives the terms to be included in the force balance.
If we define the lower fluid in the sketch above to be fluid 1 and the upper fluid to be fluid 2. The traction exerted by fluid 1 onto fluid 2, $\mathbf{t}^{[1]*}$ and that exerted by fluid 2 onto fluid 1, $\mathbf{t}^{[2]*}$. Then, balance of forces requires that

$$\mathbf{t}^{[1]*} - \mathbf{t}^{[2]*} \equiv \left[\left[\boldsymbol{\tau}^* \cdot \boldsymbol{n}^{[1]}\right]\right] = \sigma^* \, \kappa^* \, \boldsymbol{n}^{[1]} + \boldsymbol{\nabla}_s \, \sigma^*$$

where we have been explicit about the fact that the curvature is dimensional and $\kappa^* > 0$ if the centre of curvature lies inside fluid 1.
After using the non-dimensionalisation described in another tutorial, the boundary condition becomes

$$\left[\left[\boldsymbol{\tau} \cdot \boldsymbol{n}^{[1]}\right]\right] = \frac{1}{Ca} \left[\sigma \, \kappa \, \boldsymbol{n} + \boldsymbol{\nabla}_s \, \sigma\right]$$

where $\sigma = \sigma^*/\sigma_{ref}$ and $Ca = \mu_{ref} \mathcal{U}/\sigma_{ref}$, is the capillary number based on a reference viscosity and surface tension. This condition can be incorporated directly into the weak form of the momentum equations because the surface integral term in these equations, as described in another tutorial , is

$$\iint_S \boldsymbol{n} \cdot \boldsymbol{\tau} \cdot \boldsymbol{\psi}^{(F)} \, \mathrm{d}S,$$

where $\boldsymbol{\psi}^{(F)}$ are the vector test functions associated with the momentum equations. Note that it is more compact to work with vector test functions rather than the Cartesian components of a vector test function in this case. The terms in the integral are exactly those on the left-hand side of the boundary condition multiplied by the test function. Hence, they become

$$\iint_S \frac{1}{Ca} \left[\sigma \, \kappa \, \boldsymbol{n} + \boldsymbol{\nabla}_s \, \sigma\right] \cdot \boldsymbol{\psi}^{(F)} \, \mathrm{d}S.$$

Early contributors computed these terms directly, but an undesirable feature is that computation of the curvature requires taking second derivatives of the position vector, which requires a higher degree of smoothness than previously demanded. We can use the surface divergence theorem again to weaken this requirement. Firstly we must use the product rule to bring the test function into the surface divergence

$$\iint_S \frac{1}{Ca} \left[\sigma \boldsymbol{\psi}^{(F)} \cdot \kappa \, \boldsymbol{n} + \boldsymbol{\nabla}_s \cdot \left(\sigma \boldsymbol{\psi}^{(F)}\right) - \sigma \, \boldsymbol{\nabla}_s \cdot \boldsymbol{\psi}^{(F)}\right] \, \mathrm{d}S;$$

and then we can use the surface divergence theorem (1) on the first two terms to obtain

$$\int_B \frac{1}{Ca} \sigma \boldsymbol{\psi}^{(F)} \cdot \boldsymbol{m} \, \mathrm{d}l - \iint_S \frac{1}{Ca} \sigma \boldsymbol{\nabla}_s \cdot \boldsymbol{\psi}^{(F)} \, \mathrm{d}S.$$

In index notation these terms become

$$\int_B \frac{1}{Ca}\sigma\psi_i^{(F)}m_i \ \mathrm{d}l - \iint_S \frac{1}{Ca}\sigma g^{\alpha\beta}\left[\boldsymbol{g}_\alpha\right]_i\left[\boldsymbol{\psi}_{,\beta}^{(F)}\right]_i \ \mathrm{d}S,$$

where $[\boldsymbol{g}_\alpha]_i$ denotes the $i$-th component of the covariant base vector $\boldsymbol{g}_\alpha$ and $\sigma_{,\beta}$ represents the derivative $\partial\sigma/\partial\zeta^\beta$ (3). These are the terms that are implemented in `oomph-lib's` free surface elements. Note that variations in surface tension are taken into account in this formulation without the explicit need to take its surface derivative. An important observation is that the intrinsic surface coordinates can be chosen to be the local coordinates of each element so that we do not need to introduce another set of coordinates.

#### 1.1.3.2 Kinematic condition

The kinematic condition is that "particles on the surface must remain on the surface". In other words the normal velocity of the surface must equal the normal rate of change of its position with time. The condition is compactly expressed in non-dimensional form as

$$\left(\boldsymbol{U} - St\frac{\partial\mathbf{R}}{\partial t}\right)\cdot\boldsymbol{n} = 0,$$

where $St$ is the Strouhal number, see `another tutorial` for details. Note that, in general $\boldsymbol{U}$ the velocity of the surface need *not* coincide with the velocity of the fluid.
Although this equation must be satisfied the details of how exactly it is implemented depend crucially on the mesh-update strategy chosen and we make use of the C++ features of inheritance and templating to avoid code duplication, see below for details.

### 1.1.4 Surface Transport Equations

Consider a chemical species with surface concentration $\Gamma$ that is only present on the surface. It can be transported within the moving surface by the standard mechanisms of advection and diffusion, but changes in the surface area can also induce changes in its concentration. The formulation of surface transport equations has been discussed many times in the literature and the main confusion surrounds how material derivatives are taken. Unlike the conventional bulk equations the surface does not occupy every point in the domain, so one cannot simply use the standard form of the material derivative.
Rather than covering the literature, here we shall simply state, *ab initio* , the governing equations formulated by Huang, Lai & Tseng as well as Cermelli et al (2005), who claim it was established by Slattery (1972). We shall demonstrate that it is equivalent to the form stated by Wong & Rumshitski and used by Campana et al (2004), but that it leads to a simpler formulation which avoids explicit calculation of the curvature (and hence second derivatives).
The dimensionless governing equations in weak form governing the transport of a scalar quantity $\Gamma$ are

$$\iint_S \left[St\left(\frac{\partial\Gamma}{\partial t} - \dot{\boldsymbol{R}}\cdot\boldsymbol{\nabla}_S\Gamma\right) + \boldsymbol{\nabla}_S\cdot(\Gamma\boldsymbol{U}) - \frac{1}{Pe_s}\boldsymbol{\nabla}_S\cdot\boldsymbol{\nabla}_S\Gamma\right]\phi\,\mathrm{d}S = 0.$$

In the above equation the time derivative is taken at fixed `'nodes'' in the finite element formulation and the ALE-like term compensates for tangential movement of these nodes along the surface. The normal movement is enforced to be exactly the same as the surface velocity by the kinematic condition. Note that the velocity in the third term of the governing equation is the full (bulk) fluid velocity. The dimensionless quantity $Pe_s = \mathcal{UL}/D_s$ is the surface Peclet number.
The formulation of Campana et al is found by decomposing the velocity in this term into normal and tangential components.

$$\boldsymbol{\nabla}_S\cdot(\Gamma\boldsymbol{U}) = \boldsymbol{\nabla}_S\cdot(\Gamma\boldsymbol{U}_t + \Gamma U_n\boldsymbol{n}).$$

The surface gradient yields a vector that is tangential to the surface so that its inner product with the unit normal, $\boldsymbol{n}$ is zero. Thus,

$$\boldsymbol{\nabla}_S\cdot(\Gamma\boldsymbol{U}) = \boldsymbol{\nabla}_S\cdot(\Gamma\boldsymbol{U}_t) + \Gamma U_n\boldsymbol{\nabla}_S\cdot\boldsymbol{n},$$

which is the starting point for Campana *et al*'s derivation because the surface divergence of the normal may be replaced by the curvature, as described above.
Returning to our formulation we use the surface divergence theorem (1) to integrate the diffusion term and the product rule to handle the third term:

$$\iint_S \left[St\left(\frac{\partial\Gamma}{\partial t} - \dot{\boldsymbol{R}}\cdot\boldsymbol{\nabla}_S\Gamma\right) + \Gamma\boldsymbol{\nabla}_S\cdot\boldsymbol{U} + \boldsymbol{U}\cdot\boldsymbol{\nabla}_S\Gamma\right]\phi + \frac{1}{Pe_s}\boldsymbol{\nabla}_S\Gamma\cdot\boldsymbol{\nabla}_S\phi\,\mathrm{d}S - \int_B \frac{1}{Pe_s}\boldsymbol{\nabla}_S\Gamma\cdot\boldsymbol{m}\,\phi\ \mathrm{d}l = 0,$$

$$\Rightarrow \quad \iint_S \left[ St\,\frac{\partial \Gamma}{\partial t} + (\boldsymbol{U} - St\,\dot{\boldsymbol{R}})\cdot\boldsymbol{\nabla}_s\Gamma + \Gamma\boldsymbol{\nabla}_s\cdot\boldsymbol{U} \right]\phi + \frac{1}{Pe_s}\boldsymbol{\nabla}_s\Gamma\cdot\boldsymbol{\nabla}_s\phi\,\mathrm{d}S - \int_B \frac{1}{Pe_s}\boldsymbol{\nabla}_s\Gamma\cdot\boldsymbol{m}\,\phi\;\mathrm{d}l = 0,$$

The line term represents no-diffusive flux out of the system. In index notation the equations are

$$\iint_S \left[ St\,\frac{\partial \Gamma}{\partial t} + g^{\alpha\beta}[\boldsymbol{g}_\alpha]_i \left\{ (U_i - St\,\dot{R}_i)\Gamma_{,\beta} + \Gamma\,[\boldsymbol{U}_{,\beta}]_i \right\} \right]\phi + \frac{1}{Pe_s}g^{\alpha\beta}\Gamma_{,\alpha}\phi_{,\beta}\,\mathrm{d}S - \int_B \frac{1}{Pe_s}g^{\alpha\beta}[\boldsymbol{g}_\alpha]_i m_i \Gamma_{,\beta}\,\phi\;\mathrm{d}l = 0,$$

These are the equations implemented in `oomph-lib` using the definitions of surface derivatives given in Differential Operators On A Surface.

## 1.2 Implementation

We will now discuss how the discrete versions of the equations derived above are actually implemented in `oomph-lib`. The basic idea is that the equations should be implemented independently of the specific element type and mesh-update strategy and a base class oomph::FluidInterfaceElement defines the generic functionality for all fluid interface elements. The only difference between the different surface geometries are in the definitions of the surface derivative operators and these are defined in specific classes oomph::LineDerivatives (1D surface), oomph::AxisymmetricDerivatives and oomph::SurfaceDerivatives (2D surface). The final specific element is created by using a special templated class that determines the node-update strategy and takes the base class, derivative class and bulk element as template arguments.

### 1.2.1 The FluidInterfaceElement class

The template-free oomph::FluidInterfaceElement class provides storage and member functions that are common to all free-surface and interface elements. The most important functions to be aware of are:

- Storage and access functions for (pointers to) the capillary and Strouhal numbers.
- Storage for a (pointer to) an external pressure degree of freedom if the boundary is a free surface, rather than an interface.
- The virtual function
  ```
  double FluidInterfaceElement::compute_surface_derivatives(...)
  ```
  specifies how the surface gradient operators are computed.
- The function
  ```
  double FluidInterfaceElement::sigma(const Vector<double> &s_local)
  ```
  that returns the surface tension at the given local coordinate; default implementation returns 1.0.
- The function
  ```
  FluidInterfaceElement::fill_in_generic_residual_contribution_interface(..)
  ```
  that is responsible for assembling the residual and jacobian contributions corresponding to the dynamic and kinematic boundary conditions.
- The function
  ```
  FluidInterfaceElement::add_additional_residual_contributions_interface(...)
  ```
  which is called from *within* the integration loop and is used to assemble any additional surface transport equations or equations arising from different node update strategies. This function is virtual so that it can be overloaded in derived classes.
- The function
  ```
  virtual int FluidInterfaceElement::kinematic_local_eqn(...)
  ```
  that is used to specify the local equation number used for the kinematic condition, which depends on the mesh-update strategy chosen.

### 1.2.2 The LineDerivatives class

The class oomph::LineDerivatives implements the specific surface derivatives associated with a one-dimensional surface in a two-dimensional domain.The global coordinate system is Cartesian so its base vectors do not vary with the surface coordinates and $[\boldsymbol{\psi}^{(F)}_{,\beta}]_i = \psi^{(F)}_{i,\beta}$; and the contribution to each component of the momentum equation is found by setting the appropriate component $\psi^{(F)}_i = 0$ for $i = 1, 2$.

### 1.2.3 The AxisymmetricDerivatives class

The class oomph::AxisymmetricDerivatives implements the specific residuals associated with a two-dimensional surface in a three-dimensional domain, under the assumption of axisymmetry. Thus, the coordinate system is cylindrical polar $(r, z, \theta)$, but it is assumed that there are no variations in the $\theta$ direction.
It is worthwhile including the required mathematics here because the terms are not the same as in the `Line↩ Derivatives` class. Specifically, if the surface coordinates are $(\zeta^1, \zeta^2) = (s, \theta)$, the position vector to the surface is given by

$$\boldsymbol{R} = \begin{pmatrix} r(s)\cos\theta \\ r(s)\sin\theta \\ z(s) \end{pmatrix} \quad \Rightarrow \quad \boldsymbol{g}_1 = \begin{pmatrix} r'(s)\cos\theta \\ r'(s)\sin\theta \\ z'(s) \end{pmatrix}, \quad \boldsymbol{g}_2 = \begin{pmatrix} -r(s)\sin\theta \\ r(s)\cos\theta \\ 0 \end{pmatrix},$$

where $r'(s) = \partial r/\partial s$ and $z'(s) = \partial z/\partial s$. Hence,

$$g_{11} = (r')^2 + (z')^2, \quad g_{12} = g_{21} = 0, \quad g_{22} = r^2, \quad \text{and} \quad g = r^2\left[(r')^2 + (z')^2\right].$$

In our standard formulation, the vector test function is given by

$$\boldsymbol{\psi}^{(F)} = \begin{pmatrix} \psi_r^{(F)}(s)\cos\theta \\ \psi_r^{(F)}(s)\sin\theta \\ \psi_z^{(F)}(s) \end{pmatrix} \quad \Rightarrow \quad \boldsymbol{\psi}_{,1}^{(F)} = \begin{pmatrix} (\psi')_r^{(F)}(s)\cos\theta \\ (\psi')_r^{(F)}(s)\sin\theta \\ (\psi')_z^{(F)}(s) \end{pmatrix} \quad \text{and} \quad \boldsymbol{\psi}_{,2}^{(F)} = \begin{pmatrix} -\psi_r^{(F)}(s)\sin\theta \\ \psi_r^{(F)}(s)\cos\theta \\ 0 \end{pmatrix}.$$

Thus the contribution to the momentum equation (3) are

$$\int_B \frac{1}{Ca}\sigma\psi_i^{(F)} m_i \; \mathrm{d}l - \iint_S \frac{1}{Ca}\sigma\left[\frac{r'(\psi')_r^{(F)} + z'(\psi')_z^{(F)}}{(r')^2 + (z')^2} + \frac{1}{r}\psi_r^{(F)}\right] \mathrm{d}S,$$

Two separate contributions are then derived from the cases $\psi_r^{(F)} = 0$ and $\psi_z^{(F)} = 0$. The difference from the `LineDerivatives` is the final term that accounts for the azimuthal curvature. In addition, we must also multiply all terms by the additional factor of $r$ in the square-root of the determinant of the surface metric tensor.

### 1.2.4 The SurfaceDerivatives class

The class oomph::SurfaceDerivatives implements the specific residuals associated with a general two-dimensional surface in a three-dimensional domain. Once again, the global coordinate is Cartesian, so the contribution to each momentum equation is found by setting the two other components of the test function to be zero.

## 1.3 The SpineLine/Axi/SurfaceFluidInterfaceElement classes

We shall discuss the "line" version of the elements, but the others are essentially the same.
The class oomph::SpineLineFluidInterfaceElement is templated by the bulk element type, `ELEMENT`, and inherits from `FluidInterfaceElement`, LineDerivatives and `Hijacked<SpineElement<FaceGeometry<↩ ELEMENT> > >`. The hijacking is only required for imposition of contact angle boundary conditions, see `another tutorial` for more details.
Note that the use of templates to make the code generic makes it hard to read. A simplified constructor is given below and simply builds the element based on the `FaceGeometry` of the bulk element and sets the indices associated with the bulk fluid velocity components from the bulk element.

```
SpineLineFluidInterfaceElement(FiniteElement* const& element_pt,
```

If a spine method is used to update the nodal positions then the spine height is the unknown associated with the kinematic condition. Thus, the function `kinematic_local_eqn(...)` is overloaded accordingly

```
int kinematic_local_eqn(const unsigned& n)
{
  return this->spine_local_eqn(n);
```

Finally, the element calculates the geometric contributions to the jacobian using the generic functionality in `ElementWithMovingNodes`

```
void fill_in_contribution_to_jacobian(Vector<double>& residuals,
```

There are no additional contributions to the residuals or jacobian.

## 1.4 The ElasticLine/Axi/SurfaceFluidInterfaceElement classes

We shall discuss the "line" version of the elements, but the others are essentially the same.
The class oomph::ElasticLineFluidInterfaceElement is templated by the bulk element type, `ELEMENT`, and inherits from `LineDerivatives`, FluidInterfaceElement and `Hijacked<FaceGeometry<ELEMENT> >`. The hijacking is again only required for imposition of contact angle boundary conditions.
The "elastic" versions of the elements are more complicated than the "spine" versions because the kinematic condition is imposed using Lagrange multipliers, following the method described by Cairncross et al A finite element method for free surface flows of incompressible fluids in three dimensions. Part I. Boundary fitted mesh motion' (2000). These Lagrange multipliers must be added to the `Nodes` on the free surface and their introduction adds additional terms to the equations governing the bulk mesh motion.
The constructor is given below and in addition to building the element based on the `FaceGeometry` of the bulk element and setting the indices associated with the bulk fluid velocity components from the bulk element, it also adds the additional storage required for the Lagrange multipliers

```
ElasticLineFluidInterfaceElement(FiniteElement* const& element_pt,
```

The kinematic boundary condition is associated with the Lagrange multiplier and `kinematic_local_eqn(...)` is overloaded accordingly

```
/// (This is the equation for the Lagrange multiplier)
inline int kinematic_local_eqn(const unsigned& n)
{
  // Get the index of the nodal value associated with Lagrange multiplier
  return this->nodal_local_eqn(n, this->lagrange_index(n));
}
```

The element calculates the geometric contributions to the jacobian using the generic functionality in `SolidElements`

```
void fill_in_contribution_to_jacobian(Vector<double>& residuals,
                                      DenseMatrix<double>& jacobian)
{
  // Call the generic routine with the flag set to 1
  EQUATION_CLASS::fill_in_generic_residual_contribution_interface(
    residuals, jacobian, 1);

  // Call the generic finite difference routine for the solid variables
  this->fill_in_jacobian_from_solid_position_by_fd(jacobian);
}
```

The additional contributions to the residuals and jacobian arise from the Lagrange multiplier contributions to the equations that determine the position of the nodes. The essential loop is the contribution below which adds the normal traction to the governing equations of solid mechanics:

```
  // Loop over the shape functions to assemble contributions
  for (unsigned l = 0; l < n_node; l++)
  {
    // Loop over the directions
    for (unsigned i = 0; i < nodal_dimension; i++)
    {
      // Now using the same shape functions for the elastic equations,
      // so we can stay in the loop
      local_eqn = this->position_local_eqn(l, 0, i);
      if (local_eqn >= 0)
      {
        // Add in the Lagrange multiplier contribution
        residuals[local_eqn] -=
          interpolated_lagrange * interpolated_n[i] * psif(l) * J * W;

        // Do the Jacobian calculation
        if (flag)
        {
          // Loop over the nodes
          for (unsigned l2 = 0; l2 < n_node; l2++)
          {
            // Dependence on solid positions will be handled by FDs
            // That leaves the Lagrange multiplier contribution
            local_unknown = this->kinematic_local_eqn(l2);
            if (local_unknown >= 0)
            {
              jacobian(local_eqn, local_unknown) -=
                psif(l2) * interpolated_n[i] * psif(l) * J * W;
            }
          }
        } // End of Jacobian calculation
      }
    }
  } // End of loop over shape functions
```

## 1.5 Surface Transport Implementation

The recommended strategy for implementing surface transport equations is to inherit from the appropriate `Fluid↩InterfaceElement`, and include the equations independently of the mesh-update strategy by overloading the function
`FluidInterfaceElement::add_additional_residual_contributions_interface(...)`
The mesh-update specialisations can be added by further specialisation as required. Alternatively, one could simply inherit directly from the specialised element. This is the approach taken in this `example code.`
An important point is that the axisymmetric formulation of the surface divergence of a vector is not entirely trivial, as discussed in The AxisymmetricDerivatives class. In the context of surface transport the term that is always present is $\boldsymbol{\nabla}_S \cdot \boldsymbol{U}$. Using the same surface coordinates as above, $(\zeta^1, \zeta^2) = (s, \theta)$, the velocity vector is

$$\boldsymbol{U} = \begin{pmatrix} U_r(s)\cos\theta \\ U_r(s)\sin\theta \\ U_z(s) \end{pmatrix} \quad \Rightarrow \quad \boldsymbol{U}_{,1} = \begin{pmatrix} \frac{\partial U_r}{\partial s}\cos\theta \\ \frac{\partial U_r}{\partial s}\sin\theta \\ \frac{\partial U_z}{\partial s} \end{pmatrix}, \quad \boldsymbol{U}_{,2} = \begin{pmatrix} -U_r\sin\theta \\ U_r\cos\theta \\ 0 \end{pmatrix}.$$

Thus, the surface divergence term is

$$\boldsymbol{\nabla}_S \cdot \boldsymbol{U} = \frac{\left(\frac{\partial U_r}{\partial s}\frac{\partial r}{\partial s} + \frac{\partial U_z}{\partial s}\frac{\partial z}{\partial s}\right)}{(r')^2 + (z')^2} + \frac{U_r}{r},$$

which is exactly the same form as the surface divergence of the vector test function derived in the section The AxisymmetricDerivatives class, as expected.

## 1.6 PDF file

A `pdf version` of this document is available. \